

Anomaly Detection

The Gaussian Distribution, Density Estimation, Evaluation, and Choosing Features

Rokan Uddin Faruqui
Professor, Department of CSE

University of Chittagong
rufaruqui@cu.ac.bd

Week 7 · Part 2

From groups to outliers

Part 1 gave us

K-means: find groups in unlabelled data by alternating assignment and centroid updates, minimising the distortion J.

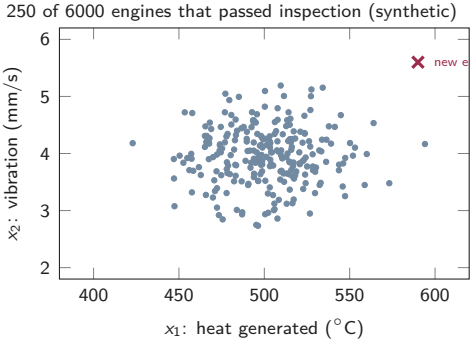
Clustering asks what the typical points look like. Anomaly detection asks the opposite: which points look *unlike* the rest? It learns what normal is from unlabelled data, then flags whatever is improbable under that model.

Today

1. Finding unusual events
2. The Gaussian (normal) distribution
3. The anomaly detection algorithm
4. Developing and evaluating a detector
5. Anomaly detection vs. supervised learning
6. Choosing what features to use

Finding Unusual Events

Example: aircraft engines



The task

Data: features $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}$ of engines already built — almost all of them fine. A new engine $\mathbf{x}_{\text{test}}$ comes off the line. Is it like the others, or should it be inspected?

Density estimation

Build a model $p(\mathbf{x})$ of how probable each feature vector is. Then

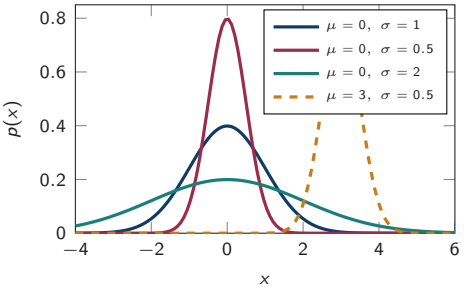
$$p(\mathbf{x}_{\text{test}}) < \varepsilon \;\Rightarrow\; \text{flag as an anomaly.}$$

Applications

Fraud detection (logins, purchases, typing speed); manufacturing (engines, circuit boards); monitoring computers in a data centre (memory, disk, CPU).

The Gaussian Distribution

The Gaussian (normal) distribution



Gaussian density

$$p(x;\mu,\sigma^2) = \frac{1}{\sqrt{2\pi}\,\sigma}\exp\!\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

μ: the centre. σ: the spread (standard deviation); σ^2 the variance.

The area under each curve is 1. Halve σ and the bell becomes half as wide and twice as tall.

A density is not a probability

With $\sigma = 0.5$ the peak is 0.798; with $\sigma = 0.1$ it is 3.99. Values above 1 are fine — only areas are probabilities.

Estimating μ and σ^2 from data

Parameter estimates

$$\mu = \frac{1}{m}\sum_{i=1}^{m} x^{(i)}, \qquad \sigma^2 = \frac{1}{m}\sum_{i=1}^{m}\left(x^{(i)} - \mu\right)^2$$

These are the maximum-likelihood estimates. Statisticians often divide by $m-1$ instead; for the thousands of examples typical here the difference is negligible.

The 6000 engines

	μ_j	σ_j
x_1, heat (°C)	499.79	25.12
x_2, vibration (mm/s)	3.993	0.495

Key idea

Fitting a Gaussian is two averages. Nothing is trained by gradient descent; there are no labels.

The Anomaly Detection Algorithm

Density estimation with n features

Anomaly detection algorithm

1. Choose features x_j that might indicate anomalies.
2. Fit μ_j, σ_j^2 for each feature $j = 1, \ldots, n$ on the training set.
3. For a new $\mathbf{x}$, compute

$$p(\mathbf{x}) = \prod_{j=1}^{n} p(x_j; \mu_j, \sigma_j^2).$$

4. Flag $\mathbf{x}$ as an anomaly if $p(\mathbf{x}) < \varepsilon$.

Multiplying the densities treats the features as statistically independent. The algorithm works well in practice even when they are not quite.

Three new engines

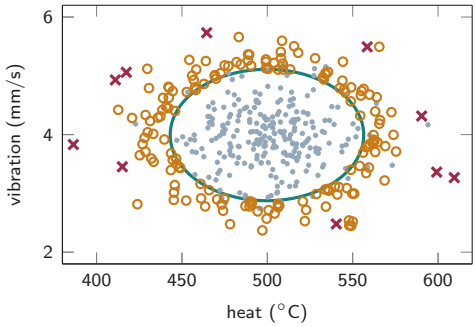
$\mathbf{x}$	$p(x_1)$	$p(x_2)$	$p(\mathbf{x})$
$(505, 4.1)$	0.0155	0.787	1.2×10^{-2}
$(560, 4.9)$	0.0009	0.151	1.4×10^{-4}
$(590, 5.6)$	0.000025	0.0042	1.0×10^{-7}

With $\varepsilon = 10^{-5}$: the first two are normal; the third, 3.6σ hot and 3.2σ shaky, is flagged.

Key idea

$p(\mathbf{x})$ is small when *any* feature is far from its mean — one improbable reading is enough.

Turning the threshold ε



teal: the boundary $p(\mathbf{x}) = \varepsilon$

×: the 10 faulty validation engines ○: good engines flagged

Validation set: 2000 good, 10 faulty

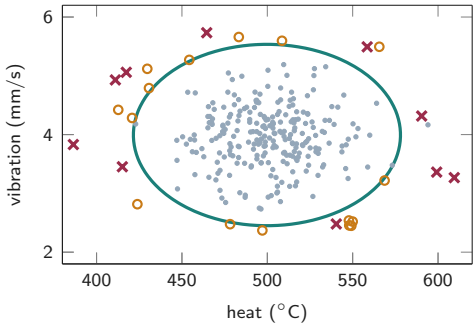
ε	TP	FP	FN	P	R	F_1
10^{-3}	10	156	0	0.060	1.0	0.114
10^{-4}	10	16	0	0.385	1.0	0.556
3×10^{-5}	9	8	1	0.529	0.9	0.667
10^{-5}	7	2	3	0.778	0.7	0.737
3×10^{-6}	3	0	7	1.000	0.3	0.462

$\varepsilon = 10^{-3}$: flags 10 of 10 faulty and 156 of 2000 good validation engines. $F_1 = 0.114$.

Key idea

The boundary of $p(\mathbf{x}) = \varepsilon$ is an ellipse (the real level set, drawn from the formula). Smaller ε widens it: fewer false alarms, more missed faults.

Turning the threshold ε



teal: the boundary $p(\mathbf{x}) = \varepsilon$

×: the 10 faulty validation engines ○: good engines flagged

Validation set: 2000 good, 10 faulty

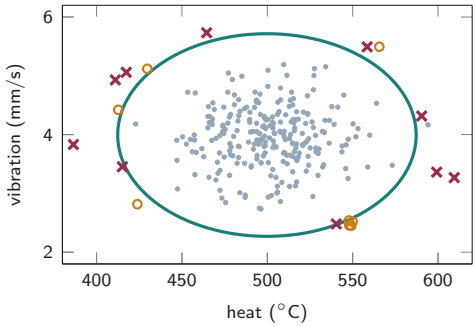
ε	TP	FP	FN	P	R	F_1
10^{-3}	10	156	0	0.060	1.0	0.114
10^{-4}	10	16	0	0.385	1.0	0.556
3×10^{-5}	9	8	1	0.529	0.9	0.667
10^{-5}	7	2	3	0.778	0.7	0.737
3×10^{-6}	3	0	7	1.000	0.3	0.462

$\varepsilon = 10^{-4}$: flags 10 of 10 faulty and 16 of 2000 good validation engines. $F_1 = 0.556$.

Key idea

The boundary of $p(\mathbf{x}) = \varepsilon$ is an ellipse (the real level set, drawn from the formula). Smaller ε widens it: fewer false alarms, more missed faults.

Turning the threshold ε



teal: the boundary $p(\mathbf{x}) = \varepsilon$

×: the 10 faulty validation engines ○: good engines flagged

Validation set: 2000 good, 10 faulty

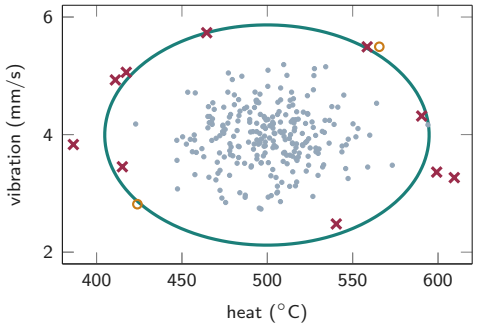
ε	TP	FP	FN	P	R	F_1
10^{-3}	10	156	0	0.060	1.0	0.114
10^{-4}	10	16	0	0.385	1.0	0.556
3×10^{-5}	9	8	1	0.529	0.9	0.667
10^{-5}	7	2	3	0.778	0.7	0.737
3×10^{-6}	3	0	7	1.000	0.3	0.462

$\varepsilon = 3 \times 10^{-5}$: flags 9 of 10 faulty and 8 of 2000 good validation engines. $F_1 = 0.667$.

Key idea

The boundary of $p(\mathbf{x}) = \varepsilon$ is an ellipse (the real level set, drawn from the formula). Smaller ε widens it: fewer false alarms, more missed faults.

Turning the threshold ε



teal: the boundary $p(\mathbf{x}) = \varepsilon$

×: the 10 faulty validation engines ○: good engines flagged

Validation set: 2000 good, 10 faulty

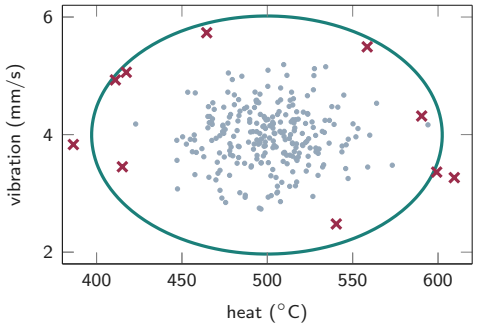
ε	TP	FP	FN	P	R	F_1
10^{-3}	10	156	0	0.060	1.0	0.114
10^{-4}	10	16	0	0.385	1.0	0.556
3×10^{-5}	9	8	1	0.529	0.9	0.667
10^{-5}	7	2	3	0.778	0.7	0.737
3×10^{-6}	3	0	7	1.000	0.3	0.462

$\varepsilon = 10^{-5}$: flags 7 of 10 faulty and 2 of 2000 good validation engines. $F_1 = 0.737$.

Key idea

The boundary of $p(\mathbf{x}) = \varepsilon$ is an ellipse (the real level set, drawn from the formula). Smaller ε widens it: fewer false alarms, more missed faults.

Turning the threshold ε



teal: the boundary $p(\mathbf{x}) = \varepsilon$

×: the 10 faulty validation engines ○: good engines flagged

Validation set: 2000 good, 10 faulty

ε	TP	FP	FN	P	R	F_1
10^{-3}	10	156	0	0.060	1.0	0.114
10^{-4}	10	16	0	0.385	1.0	0.556
3×10^{-5}	9	8	1	0.529	0.9	0.667
10^{-5}	7	2	3	0.778	0.7	0.737
3×10^{-6}	3	0	7	1.000	0.3	0.462

$\varepsilon = 3 \times 10^{-6}$: flags 3 of 10 faulty and 0 of 2000 good validation engines. $F_1 = 0.462$.

Key idea

The boundary of $p(\mathbf{x}) = \varepsilon$ is an ellipse (the real level set, drawn from the formula). Smaller ε widens it: fewer false alarms, more missed faults.

Developing and Evaluating a Detector

Real-number evaluation, with a few labels

The engines: 10 000 good, 20 faulty

set	good	faulty
training	6000	0 (assumed)
cross-validation	2000	10
test	2000	10

Train on unlabelled, mostly-normal data. Use the few labelled anomalies only to choose ε (and features), then to report.

A few anomalies slipping into the training set is harmless: they barely move μ and σ.

Procedure

1. Fit $p(\mathbf{x})$ on the training set.
2. On validation, predict $y = 1$ if $p(\mathbf{x}) < \varepsilon$.
3. Classes are skewed (10 in 2010): measure P, R, F_1 (Week 6), not accuracy.
4. Keep the best-F_1 ε: 10^{-5}, $F_1 = 0.737$.

Test set, once

At $\varepsilon = 10^{-5}$: TP $= 8$, FP $= 0$, FN $= 2$; precision 1.0, recall 0.8, $F_1 = 0.889$.

Very few anomalies

With only 2 faults, use both for validation and skip the test set — the result is then optimistic.

Anomaly Detection vs. Supervised Learning

If we have some labels, why not train a classifier?

Anomaly detection	Supervised learning
Very few positives (0–20 is common), many negatives.	Enough positives and negatives to learn both.
Many *different kinds* of anomaly; future ones may look like none seen so far.	Future positives are likely to resemble the training positives.
Fraud, where attackers invent new tricks; a never-before-seen engine fault; a misbehaving server.	Spam (the types recur); known manufacturing defects such as scratches; weather; diagnosing a known disease.

The difference in one line

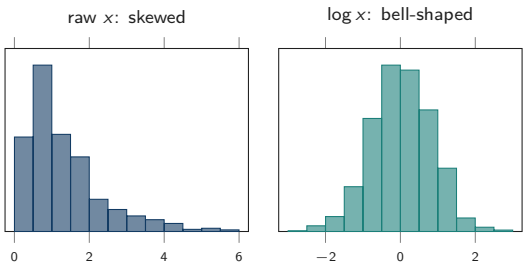
A classifier learns what positives look like. An anomaly detector learns what *normal* looks like, and flags anything else.

In practice

Many systems use both: a classifier for the known fraud patterns, an anomaly detector for the unknown ones.

Choosing What Features to Use

Make features look Gaussian



1000 synthetic readings; the raw histogram omits 14 values above 6.

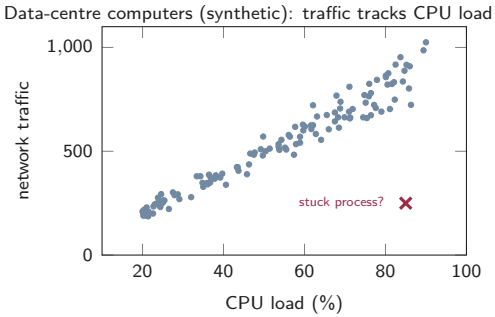
Why it matters

The algorithm models each feature as a Gaussian. A long right tail makes large but perfectly normal values look improbable, and squeezes the real outliers together.

Transformations to try

$\log x$, $\log(x + c)$, $\sqrt{x}$, $x^{1/3}$. Plot the histogram, choose by eye, and apply the *same* transformation to the validation and test sets.

Error analysis: invent the feature that exposes the miss



The miss

High CPU with little traffic: a process stuck in a loop. Each feature alone is unremarkable (1.5σ and -1.4σ), so $p(\mathbf{x}) = 4.5 \times 10^{-6}$ — lower than for 79 of the 500 normal machines. No ε flags it without flagging those 79.

Add a feature

$x_3 = \dfrac{\text{CPU load}}{\text{network traffic}}$. Normal machines: 0.101 ± 0.009. The stuck one: 0.34, which is 28σ out. Now $p(\mathbf{x}) \approx 10^{-174}$, against at least 1.1×10^{-7} for every normal machine.

Error analysis for anomaly detection: find the anomalies $p(\mathbf{x})$ missed, ask what makes them unusual, and turn that into a feature.

Wrap-up and Assessment

Summary

What to remember

- Model normal data with $p(\mathbf{x})$; flag $p(\mathbf{x}) < \varepsilon$.
- Fit one Gaussian per feature: μ_j and σ_j^2 are averages.
- Train on unlabelled data; use a few labelled anomalies to choose ε by F_1, and to test.
- Prefer anomaly detection when positives are rare and varied; supervised learning when they are plentiful and alike.
- Transform skewed features towards a bell shape; build new features from the anomalies you miss.

Formulas

$$p(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/2\sigma^2}$$

$$p(\mathbf{x}) = \prod_{j=1}^{n} p(x_j; \mu_j, \sigma_j^2)$$

Next

Week 8: recommender systems — collaborative filtering and content-based filtering.

Descriptive questions

1. Explain anomaly detection by density estimation, and give three applications. **[4]**
2. State the Gaussian density. Sketch it for $(\mu, \sigma) = (0, 1)$, $(0, 0.5)$ and $(3, 1)$, and explain why its value can exceed 1. **[6]**
3. Give the estimates of μ and σ^2 from m examples, and compute them for $x = \{2, 4, 4, 4, 5, 5, 7, 9\}$. **[5]**
4. State the anomaly detection algorithm for n features. What assumption does the product make? **[5]**
5. Using the lecture's engine parameters, compute $p(\mathbf{x})$ for $\mathbf{x} = (450, 3.0)$ and decide whether it is an anomaly at $\varepsilon = 10^{-5}$. **[8]**
6. Explain how the data are split to develop an anomaly detector, and why accuracy is a poor way to choose ε. **[6]**

Descriptive questions (continued)

7. From the validation table, explain why F_1 is best at $\varepsilon = 10^{-5}$ rather than at the largest or smallest ε. **[5]**
8. Compare anomaly detection with supervised learning, and classify: (a) detecting new kinds of credit-card fraud; (b) filtering spam; (c) finding dented car panels in a factory with 5000 labelled dents. **[6]**
9. Why should features be roughly Gaussian? Name three transformations, and explain why the same one must be applied to every data split. **[5]**
10. Explain error analysis for anomaly detection using the stuck-process example. **[6]**
11. Show that the level set $p(\mathbf{x}) = \varepsilon$ for two features is an ellipse, and find its semi-axes for the engines at $\varepsilon = 10^{-5}$. **[8]**
12. Why do a few anomalies in the training set do little harm? **[4]**

Design questions

1. **Design a fraud detector.** A mobile financial service in Bangladesh wants to flag unusual transactions. Design the features (including at least one ratio), the training data, how you will obtain labelled frauds for validation, and how you choose ε given the cost of a blocked genuine payment. **[12]**
2. **Design for a garment factory.** Design an anomaly detector for sewing machines using vibration and power sensors, including how you would detect a machine that drifts slowly over weeks. **[10]**
3. **Design the decision.** You have 40 labelled defects and 50 000 good circuit boards. Design an experiment to decide between anomaly detection and a supervised classifier. **[12]**
4. **Design a feature-transform check.** Design an automatic procedure that picks a transformation for each feature before fitting $p(\mathbf{x})$, and a test that it improves validation F_1. **[10]**

Reading

Primary text

G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.

→ **§4.4.1–4.4.2** — Gaussian densities and their parameter estimates; **§4.4.2** — thresholds, sensitivity and specificity.

Supplementary

C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

→ **§1.2.4** — the Gaussian distribution and its maximum-likelihood estimates; **§2.3** — the multivariate Gaussian, whose level sets are ellipses.

Lab (CSE 816, Week 7): anomaly detection.